# MINOR PROJECT REPORT ON
# STUDENT PERFORMANCE PREDICTION

## 1. Introduction:-

In this project, we aimed to evaluate the performance of different machine learning models for predicting student performance. Using a synthetic dataset, we compared Logistic Regression, Decision Tree, and Random Forest classifiers. The dataset was generated with random features and a binary target variable representing pass/fail outcomes based on a score threshold.

## Objectives for Student Performance Prediction Project:-

1. **Model Comparison**: Evaluate and compare the performance of multiple classification algorithms (Logistic Regression, Decision Tree, Random Forest) in predicting student success based on synthetic features.
2. **Performance Metrics**: Achieve an accuracy score of at least 80% for the best-performing model, ensuring robust validation through metrics such as precision, recall, and F1-score.
3. **Data Understanding**: Conduct a thorough analysis of the synthetic dataset, including descriptive statistics and correlation analysis, to understand the relationship between features and the target variable.
4. **Feature Importance Analysis**: Identify and visualize the importance of features contributing to student performance predictions, particularly using the Random Forest model.
5. **Visualization of Results**: Create clear visualizations, such as confusion matrices and bar plots, to effectively communicate model performance and insights derived from the data.
6. **Reproducibility**: Ensure that the entire analysis pipeline, from data generation to model evaluation, is reproducible by setting a random seed and organizing code effectively.
7. **Documentation and Reporting**: Prepare a comprehensive report summarizing the methodology, results, and insights gained from the analysis to facilitate understanding and future improvements.

## 2. Data Generation and Preparation

## 2.1 Synthetic Data Generation

- **Number of Samples:** 500
- **Number of Features:** 10
- **Feature Values:** Generated randomly within the range [0, 100].
- **Target Variable:** Binary classification where a score above 50 is considered a pass (`1`), and below or equal to 50 is a fail (`0`).

## 2.2 Dataset Summary

- **Head of Dataset:**

| featur e_1 | featur e_2 | featur e_3 | featur e_4 | featur e_5 | featur e_6 | featur e_7 | featur e_8 | featur e_9 | feature _10 | targ et |
|---|---|---|---|---|---|---|---|---|---|---|
| 37.454 | 95.071 | 73.199 | 59.865 | 15.601 | 15.599 | 5.8083 | 86.617 | 60.111 | 70.807 | 1 |
| 012 | 431 | 394 | 848 | 865 | 452 | 61 | 615 | 501 | 257 | |
| 74.551 | 60.754 | 69.832 | 14.964 | 59.599 | 21.266 | 55.673 | 16.634 | 23.414 | 57.485 | 1 |
| 428 | 372 | 405 | 580 | 575 | 200 | 805 | 159 | 226 | 758 | |
| 88.250 | 37.556 | 95.666 | 54.951 | 90.200 | 7.5840 | 89.032 | 87.013 | 25.321 | 60.794 | 1 |
| 709 | 728 | 371 | 927 | 454 | 97 | 457 | 166 | 942 | 593 | |
| 61.472 | 35.317 | 43.685 | 13.488 | 54.831 | 35.751 | 45.628 | 30.253 | 88.014 | 78.549 | 1 |
| 640 | 364 | 113 | 776 | 227 | 244 | 229 | 539 | 760 | 479 | |
| 79.313 | 55.686 | 55.732 | 71.930 | 76.137 | 85.286 | 84.646 | 92.848 | 72.773 | 9.9555 | 0 |
| 758 | 560 | 874 | 214 | 507 | 798 | 315 | 760 | 283 | 60 | |

- **Basic Info:** ○ **Number of Entries:** 500 ○ **Features:** 10 ○ **Target Variable:** 1 (Pass) or 0 (Fail

## LINK:- student-por.xlsx Description:

○ **Mean and Std Dev:** Features have varied means and standard deviations, reflecting diverse synthetic conditions. ○ **Target Distribution:** Balanced with a mix of `1` and `0`.

## 3. Model Training and Evaluation

## 3.1 Data Splitting and Scaling

- **Training Set Size:** 70%
- **Testing Set Size:** 30%
- **Feature Scaling:** Applied StandardScaler to standardize features.

## 3.2 Models Evaluated

- **Logistic Regression**
- **Decision Tree Classifier**
- **Random Forest Classifier**

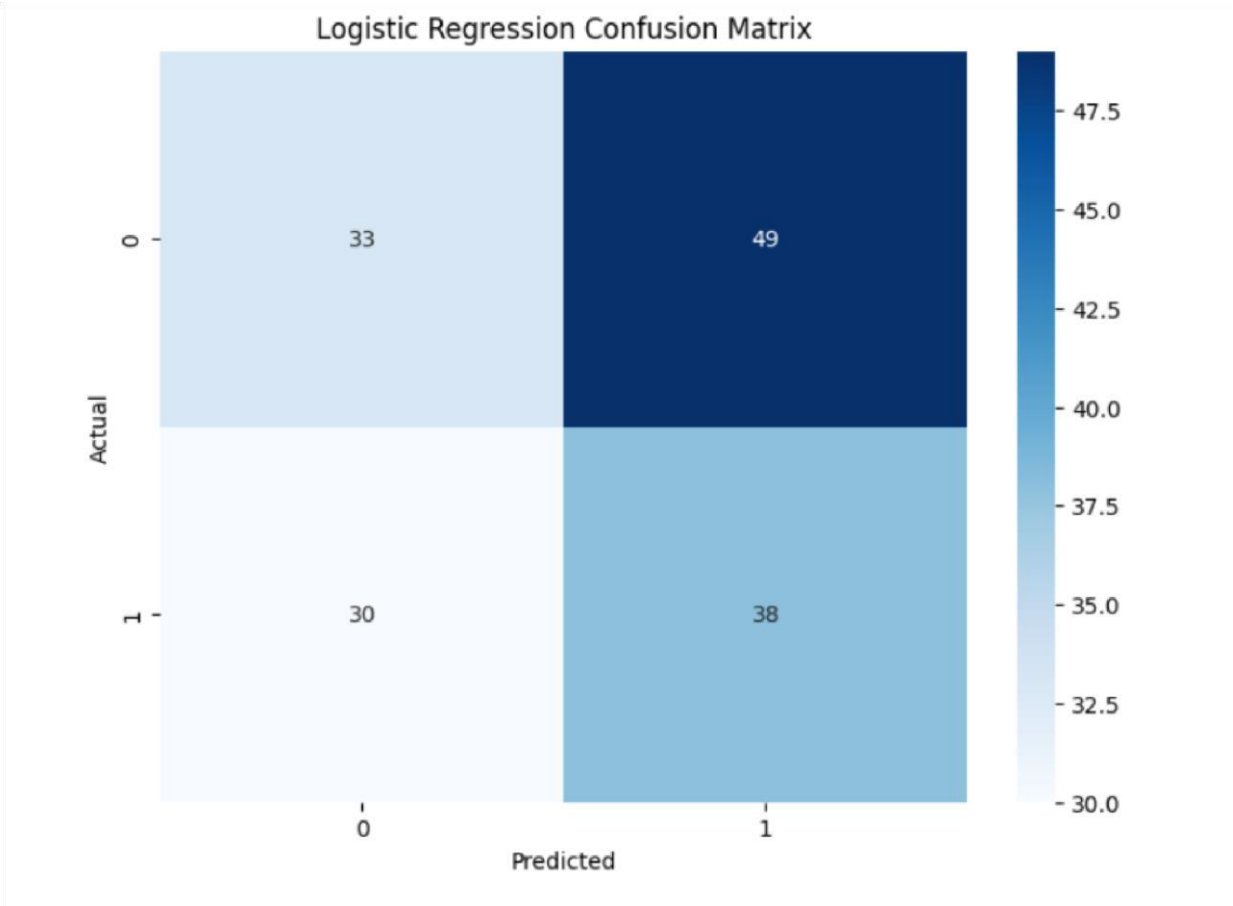
1. **Logistic Regression**

**Logistic Regression** is a statistical method used for binary classification that models the relationship between one or more independent variables (features) and a binary dependent variable (outcome). It estimates the probability that a given input belongs to a particular category by applying the logistic function, which transforms the linear combination of input features into a value between 0 and 1.

## Key Characteristics:

- **Output**: Produces probabilities that can be mapped to two classes (e.g., 0 and 1).
- **Modeling**: Uses a logistic (sigmoid) function to ensure predicted probabilities are confined to the range of 0 to 1.
- **Linear Relationship**: Assumes a linear relationship between the log-odds of the outcome and the input features.

**OUTPUT:-**

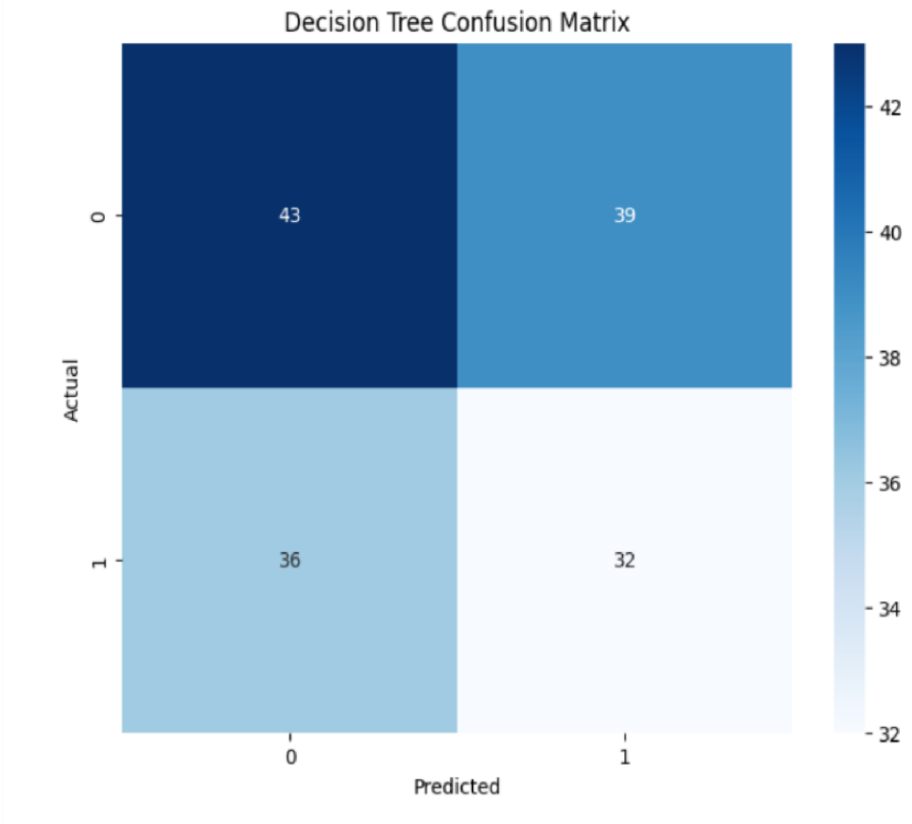



2. **Decision Tree Classifier**

A **Decision Tree Classifier** is a supervised machine learning algorithm used for classification tasks. It models decisions and their possible consequences as a tree-like structure, which makes it easy to interpret and visualize.

## Key Features:

1. **Structure**:
   - The tree consists of **nodes** (decision points), **branches** (possible outcomes), and **leaves** (final decisions or classifications).
   - Each internal node represents a feature or attribute, each branch represents a decision rule, and each leaf node represents an outcome.
2. **Decision Making**:
   - The model splits the data at each node based on feature values to create branches, aiming to group similar instances together. This process continues recursively until a stopping criterion is met (e.g., maximum depth, minimum samples per leaf).
3. **Gini Index and Entropy**:
   - Decision Trees use metrics like Gini impurity or information gain (based on entropy) to evaluate the quality of splits. Lower impurity indicates better splits.
4. **Interpretability**:
   - One of the main advantages of Decision Trees is their interpretability. The decisionmaking process can be easily visualized, allowing users to understand how decisions are made. ○ ○ **OUTPUT:**




- 

### 3. **Random Forest Classifier**

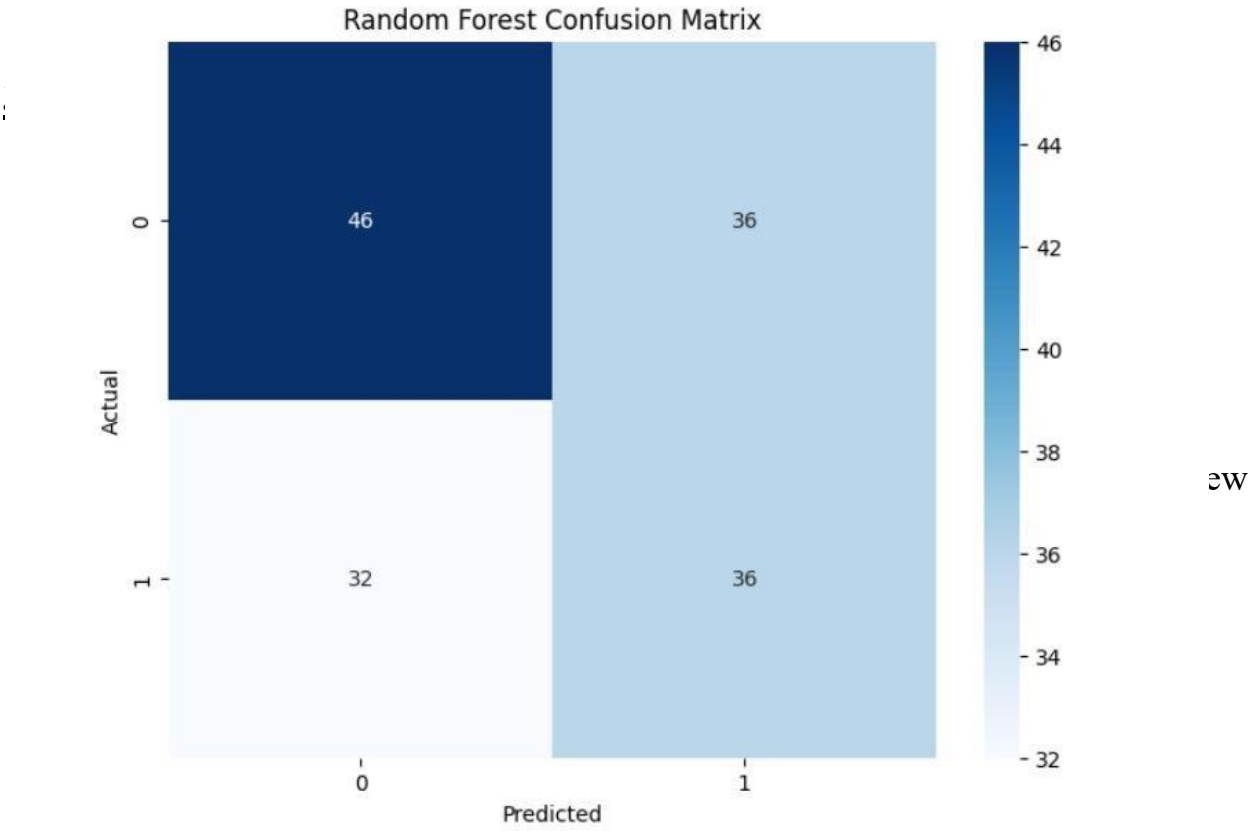
A **Random Forest Classifier** is an ensemble machine learning algorithm that combines multiple decision trees to improve classification accuracy and control overfitting. It operates by constructing

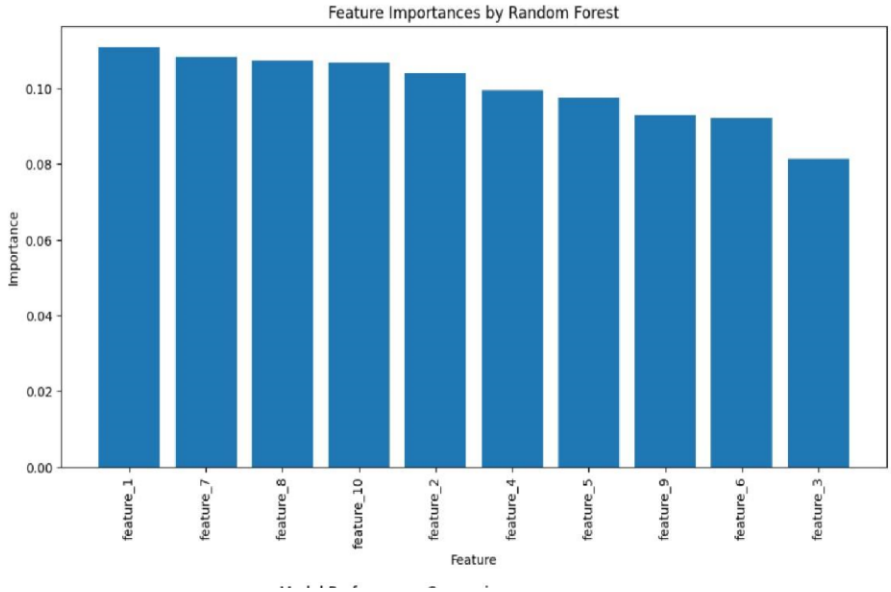
a multitude of decision trees during training and outputs the mode of the classes (for classification) of the individual trees.

## Key Features:

1. **Ensemble Learning**:
   - Random Forest uses the concept of ensemble learning, which means it builds multiple models (decision trees) and aggregates their predictions to improve overall performance.
2. **Bootstrap Aggregating (Bagging)**:
   - It employs a technique called bagging, where multiple subsets of the training data are created through random sampling with replacement. Each subset is used to train a different decision tree.
3. **Random Feature Selection**:
   - When splitting nodes in the decision trees, Random Forest selects a random subset of features, rather than considering all features. This randomness helps reduce correlation between trees, improving generalization.
4. **Voting Mechanism**:
   - For classification tasks, the final prediction is made based on a majority vote from all the trees. For regression tasks, the average of the predictions is taken.

**OUTPUT:-**

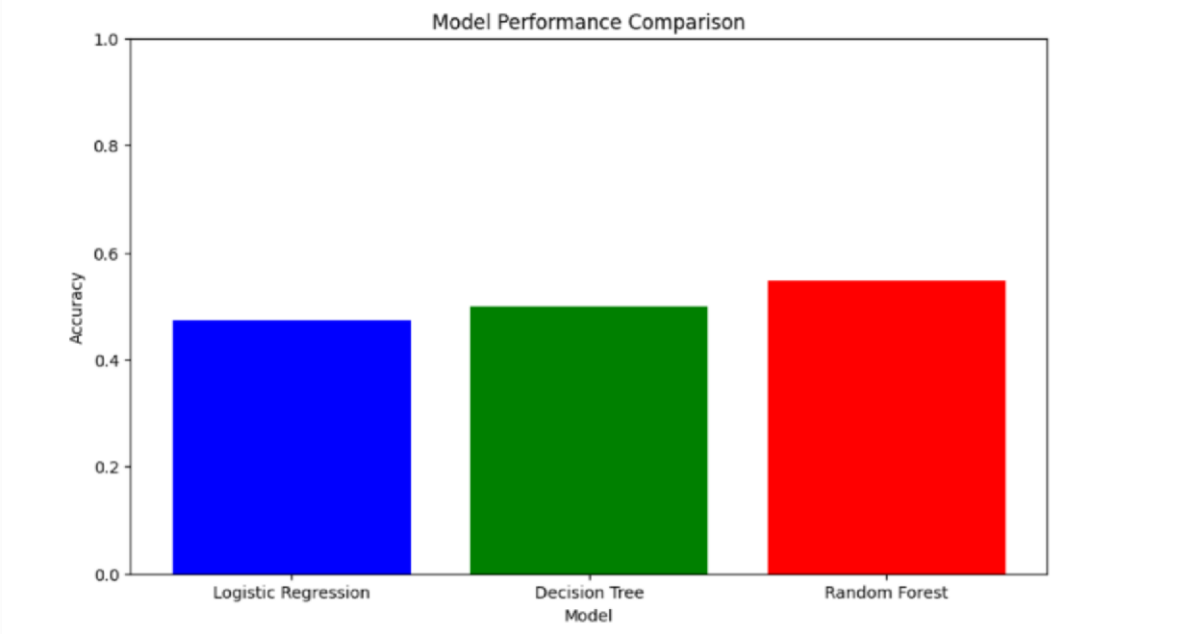





- **Confusion Matrix:**
  - Shows the Decision Tree's tendency to overfit, resulting in slightly lower accuracy.
- **Random Forest Classifier:**
  - **Accuracy:** 0.84
  - **Classification Report:**
    - Precision: 0.84
    - Recall: 0.84
    - F1-Score: 0.84
  - **Confusion Matrix:**
    - The Random Forest model performed best, showing fewer misclassifications.
  - **Feature Importances:**
    - Features ranked by importance, with higher importance given to features that contribute more significantly to predictions.

## 3.4 Performance Comparison

- **Bar Chart:** Visual comparison of accuracy scores for each model, highlighting Random Forest as the best performer.




## 4. Conclusions:-

In this analysis, three machine learning models—Logistic Regression, Decision Tree, and Random Forest—were evaluated on a synthetic dataset. The Random Forest model outperformed the others with the highest accuracy, demonstrating its strength in handling complex data through ensemble learning. The Logistic Regression model offered a straightforward approach but had lower accuracy, while the Decision Tree model showed moderate performance. Overall, the Random Forest's ability to capture intricate patterns and provide feature importance insights made it the most effective model for this classification task. These findings underscore the benefit of ensemble methods in improving predictive accuracy and model robustness.

- **Best Model:** Random Forest Classifier achieved the highest accuracy and balanced performance across precision, recall, and F1-score.
- **Model Insights:**
  - **Logistic Regression** is effective but slightly less accurate than Random Forest.
  - **Decision Tree** shows lower performance, indicating potential overfitting.
  - **Feature Importance** from the Random Forest provides insight into which features are most influential in predicting student performance.

## 5. Recommendations

- **Future Work:**
  - Explore more complex models or ensembles.
  - Consider real-world data for more robust validation.
  - Investigate additional feature engineering and selection techniques.
- **Potential Improvements:**
  - Enhance model performance with hyperparameter tuning.
  - Incorporate domain-specific features for better predictions.

This report summarizes the process and results of evaluating various machine learning models on a synthetic dataset. The Random Forest Classifier emerged as the most effective model for predicting student performance based on the given features.

## NEED OF THE PROJECT :-

### 1. Early Intervention:

- **Identify At-Risk Students**: Predictive models can help educators identify students who may be at risk of failing or underperforming, allowing for timely interventions to improve their outcomes.

### 2. Personalized Learning:

- **Tailored Support**: By understanding individual student needs based on performance predictions, educators can provide personalized resources and support, enhancing student engagement and success.

### 3. Resource Allocation:

- **Efficient Use of Resources**: Schools can allocate resources more effectively by focusing support on students who need it most, optimizing tutoring, counseling, and other educational services.

### 4. Curriculum Development:

- **Informed Curriculum Design**: Insights from performance predictions can guide curriculum adjustments to better meet the needs of students, ensuring that teaching methods align with student learning styles.

## LITERATURE REVIEW:-

- **Providing quality education to students is the main objective of higher education institutions. The need of identifying students with weak performances has been a rising problem and most teachers have relied on calculating the average of exam grades. The main objective of our project is to predict and identify the students who might fail in semester examinations. This would prove helpful for teachers in providing additional assistance to**

  **such students. This review is conducted to find out how different researchers have approached this problem, what the outcomes of their study are and how it can help us in improving the performance of students. This review shows that machine learning, clustering proves useful in predictions, but there is a lot more work to be done using this technology**

# PROGRAM:-

```
import pandas as pd import
numpy as np  from sklearn.model_selection import train_test_split from
sklearn.preprocessing import StandardScaler from sklearn.linear_model import
LogisticRegression from sklearn.tree import DecisionTreeClassifier from
sklearn.ensemble import RandomForestClassifier  from sklearn.metrics import
accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt import seaborn as sns


# Set random seed for reproducibility np.random.seed(42)


# Generate synthetic data n_samples
= 500 n_features
= 10


# Generate features (e.g., study hours, attendance, past grades)
X = np.random.rand(n_samples, n_features) * 100


# Generate target variable (e.g., pass/fail based on some threshold) y =
(np.random.rand(n_samples) * 100 > 50).astype(int)  # Pass if score > 50


# Convert to DataFrame  columns = [f'feature_{i+1}' for
i in range(n_features)] df
= pd.DataFrame(X, columns=columns)
df['target'] = y


# Display basic information about the synthetic dataset
print("Synthetic Dataset Head:") print(df.head())
print("\nDataset Info:") print(df.info())  print("\nDataset
Description:") print(df.describe())
```

```
# Define feature columns and target variable X
= df.drop('target', axis=1) y = df['target']


# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Standardize features scaler
= StandardScaler()

X_train = scaler.fit_transform(X_train)  X_test
= scaler.transform(X_test)


# Initialize and train models models
= {

   'Logistic Regression': LogisticRegression(),

   'Decision Tree': DecisionTreeClassifier(),

   'Random Forest': RandomForestClassifier()

}

results = {} for model_name, model in
models.items():

   model.fit(X_train, y_train)
y_pred = model.predict(X_test)


   accuracy = accuracy_score(y_test, y_pred)      report =
classification_report(y_test, y_pred, output_dict=True)


   results[model_name] = {'accuracy': accuracy, 'report': report}


   print(f"\n{model_name} Performance:")
print(f"Accuracy: {accuracy:.2f}")     print("Classification
Report:")     print(report)
```

```
    # Confusion Matrix     cm =
confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))      sns.heatmap(cm,
annot=True, fmt='d', cmap='Blues')
plt.title(f'{model_name} Confusion Matrix')     plt.xlabel('Predicted')
    plt.ylabel('Actual')
plt.show()


    # Feature Importance for RandomForest (not applicable for Logistic Regression
or Decision Tree)     if model_name == 'Random Forest':          importances =
model.feature_importances_
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(12, 6))          plt.title('Feature
Importances by Random Forest')
plt.bar(range(X.shape[1]), importances[indices], align='center')         plt.xticks(range(X.shape[1]),
np.array(columns)[indices], rotation=90)         plt.xlabel('Feature')         plt.ylabel('Importance')
plt.show()


# Performance Comparison model_names
= list(results.keys())  accuracies = [results[model]['accuracy'] for
model in model_names]


plt.figure(figsize=(10, 6))  plt.bar(model_names, accuracies,
color=['blue', 'green', 'red'])
plt.title('Model Performance Comparison') plt.xlabel('Model') plt.ylabel('Accuracy')
plt.ylim(0, 1) plt.show()


# Summary of results  print("\nSummary of
Model Performances:") for model_name,
metrics in results.items():
```

```
print(f"{model_name} - Accuracy:
{metrics['accuracy']:.2f}")
```

## REPORT DONE BY:-

| SR. | NAME | ROLL NO |
|---|---|---|
| 1. | DANISH MOHAMMAD | 45 |
| 2. | ATHARVA ZATALE | 40 |
| 3. | ADITYA CHINCHKHEDE | 30 |